GABRIEL POPA

Senior Software Engineer - Node (ExpressJS/ NestJS) • Azure
Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | https://www.linkedin.com/in/gabriel-popa-12890b414/

ABOUT ME

Senior Software Engineer with **11+ years** of experience building **backend platforms**, API-first services, and event-driven systems for **SaaS, analytics, and multi-tenant business applications**, with strongest depth in **Node.js v12–v20**, **TypeScript v4–v5**, **NestJS**, **Express**, **PostgreSQL**, **Redis**, **Kafka**, **RabbitMQ**, and **AWS**. Known for modernizing legacy services, improving stability under production load, solving memory leaks, timeout failures, duplicate processing, and data consistency issues, while delivering secure and maintainable backend architectures with strong observability, contract discipline, and safer release pipelines.

SKILLS

- **Backend & APIs: Node.js v12–v20**, **TypeScript v4–v5**, **NestJS**, **Express.js**, REST API design, GraphQL, WebSockets, background workers, job queues, API versioning, middleware design, OpenAPI/Swagger, JWT, OAuth2, RBAC, microservice collaboration patterns
- **Data & Messaging: PostgreSQL v11–v15**, MySQL, Redis, Kafka, RabbitMQ, Elasticsearch, OpenSearch, schema design, query tuning, indexing strategies, transaction handling, idempotency, caching strategies, asynchronous processing, event-driven workflows
- **Cloud & DevOps: Microsoft Azure (Azure Functions, App Services, Azure AD / Entra ID, Azure Storage, Azure Monitor)**, Docker, Nginx, GitHub Actions, GitLab CI, CI/CD pipelines, Linux, containerized deployments, release automation, environment configuration, infrastructure collaboration
- **Observability / Quality / Security:** Structured logging, **OpenTelemetry**, Sentry, Prometheus, Grafana, Supertest, Jest, integration testing, contract testing, end-to-end API validation, rate limiting, audit logging, OWASP-aware practices, secret rotation, failure monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania *(Jan2024 – Dec 2025)*

- Led backend development for a multi-tenant platform using **Node.js v18–v20**, **NestJS**, **TypeScript**, and **PostgreSQL**, designing modular services that supported billing, tenant administration, reporting, and real-time operational workflows with clearer domain ownership.
- Modernized an aging backend from older **Node.js v14** runtime patterns into a safer **Node.js v20** structure through phased compatibility checks, dependency review, and controlled rollout planning, which reduced upgrade risk and improved long-term maintainability.
- Designed versioned **REST APIs** and internal service contracts that gave frontend and integration teams predictable payload behavior, while reducing breaking-change incidents during parallel feature delivery across multiple product streams.
- Resolved a production **memory leak** that gradually destabilized long-running worker processes by tracing stale listeners, reviewing heap growth, and tightening cache lifecycle handling, which restored steady runtime behavior under sustained traffic.
- Solved duplicate billing execution issues by implementing **idempotency keys**, improving transaction boundaries, and reinforcing **PostgreSQL** constraints, which reduced double-processing risk and improved financial workflow integrity by **33%**.
- Implemented asynchronous event handling with **Kafka** and **RabbitMQ** for notifications, billing events, and background reconciliation, which reduced direct service coupling and improved recovery behavior when downstream dependencies became slow or unavailable.
- Strengthened authentication and authorization flows with **JWT**, **OAuth2**, **RBAC**, audit logging, and request throttling, which improved access control for tenant-sensitive operations and reduced manual investigation effort during permission disputes by **24%**.

- Improved slow reporting endpoints by reviewing query plans, restructuring expensive joins, and introducing selective pagination and caching, which stabilized response times for high-volume dashboards and reduced timeout-related complaints by **38%**.
- Built resilient background workers for export generation and reconciliation flows, replacing request-bound processing that previously failed on large payloads, which improved reliability for heavy operational jobs and reduced failed exports by **41%**.
- Introduced **OpenAPI** contract checks and API validation into **GitHub Actions** pipelines, allowing frontend and backend changes to move more independently while reducing integration regressions during active parallel development by **27%**.
- Expanded production visibility using structured logs, **OpenTelemetry**, **Prometheus**, **Grafana**, and **Sentry**, making it easier to trace failures across request paths, worker jobs, and external dependencies during incident response.
- Containerized backend services with Docker and supported deployment flows across Azure-based environments, improving release consistency, environment parity, and rollback confidence while reducing deployment-related issues observed after hotfix releases by **29%.**

Software Engineer

Next Apps | Ghent, Belgium *(Nov2020 – Nov 2023)*

- Built maintainable backend services with **Node.js v14–v18**, **Express.js**, and later **NestJS**, supporting customer-facing product workflows with cleaner routing boundaries, stronger validation handling, and more predictable service-layer organization.
- Led a practical migration from legacy **Express.js** structure into **NestJS** modules and providers, which improved testability, reduced controller bloat, and made business rules easier to extend without destabilizing surrounding endpoints.
- Implemented versioned **REST endpoints** with deprecation-safe patterns, allowing consumer applications to adopt new fields and workflows gradually while reducing rollout friction across internal and external integrations.
- Improved high-traffic API performance with **Redis** caching, explicit invalidation rules, and TTL strategies, which reduced database pressure during peak usage windows while preserving freshness for frequently requested records by **31%**.
- Resolved recurring **502** and **504** production failures by tuning **Nginx** timeout behavior, profiling slow handlers, and reducing long-running query paths, which materially improved platform stability during heavier customer activity.
- Fixed a difficult data consistency problem caused by out-of-order background events by enforcing ordering-aware handling and adding reconciliation jobs, which improved downstream accuracy for stateful workflows and reduced correction effort by **26%**.
- Strengthened asynchronous job execution with retry policies, dead-letter handling, and better monitoring around failed worker states, making stuck or partially processed tasks easier to detect and recover before customer impact expanded.
- Improved authentication hardening by tightening token validation, refining secret rotation procedures, and applying request rate limiting, which reduced abuse exposure and improved confidence around backend entry points serving public-facing traffic.
- Introduced correlation IDs and structured logging across services so user-reported failures could be followed from ingress through application and worker layers, reducing mean time to isolate backend issues by **22%**.
- Stabilized CI/CD quality gates by enforcing test coverage expectations, adding API integration checks, and documenting rollback paths for riskier releases, which improved backend release confidence and reduced emergency post-release fixes by **19%**.
- Worked closely with frontend and product teams to shape new features around realistic backend constraints, which helped avoid over-coupled API designs and improved implementation speed for cross-team delivery without sacrificing maintainability.

Software Developer

Datamart | Stockholm, Sweden *(Feb 2016 – Sept 2020)*

- Built backend services for analytics and reporting workflows using **Node.js**, **Express**, **PostgreSQL**, and **Redis**, supporting large dataset retrieval, export generation, and internal operational tools with server-side pagination and caching.

- Supported refactoring of a monolithic API into smaller service-aligned modules behind a shared gateway pattern, which improved code ownership, reduced regression risk, and made the platform easier to evolve as usage expanded.
- Improved reporting speed by investigating slow SQL execution plans, adding missing indexes, and reducing heavy join costs in **PostgreSQL**, which lowered query latency on reporting-heavy screens by **41%** and improved platform usability.
- Resolved a high-impact date accuracy issue by identifying inconsistent timezone parsing between API, UI, and database layers, then standardizing serialization and storage rules to restore reporting trust for downstream business users.
- Implemented background job processing for heavy file exports so long-running work no longer blocked request-response cycles, which reduced timeout failures and gave users clearer progress visibility for large operational downloads.
- Added automated backend integration tests around critical reporting and update paths, which reduced regressions during frequent release cycles and made refactoring business rules safer for the wider engineering team.
- Improved CI reliability by separating quick unit checks from slower integration suites, reducing flaky build outcomes and giving developers faster feedback when backend changes affected shared platform behavior.
- Introduced audit logging for sensitive update flows so operations teams could trace record changes and user actions more clearly, which improved root-cause analysis during support escalations and compliance-style reviews.
- Reduced backend resource waste by reviewing unused dependencies, tightening query response shapes, and simplifying heavy transformation logic, which improved service efficiency and reduced avoidable compute overhead by **18%**.
- Supported safer release practices with smoke checks and staged deployment verification, helping identify configuration mismatches before they affected production users and reducing rollback frequency during active delivery periods.

Junior Software Developer

Berg Software | Timișoara, Romania *(Sep2014 – Mar 2016)*

- Built early backend modules using **Node.js** and **Express**, creating **REST** endpoints with consistent response structures, basic validation handling, and safer error behavior for internal administrative and workflow-oriented web applications.
- Improved older web workflows by strengthening server-side validation and aligning backend responses with front-end expectations, which reduced avoidable user input failures and lowered support noise around inconsistent form behavior by **28%**.
- Helped solve a recurring logout problem by fixing cookie and session configuration mismatches between browser and server behavior, which made authentication flow more predictable for normal users across routine daily usage.
- Developed shared helper utilities and reusable backend patterns that reduced duplicated controller logic, making smaller maintenance changes easier to apply consistently across related service endpoints and support tasks.
- Improved page and API responsiveness by enabling compression, reducing blocking middleware behavior, and tightening request handling paths, which helped older application areas feel more stable under modest traffic growth.
- Supported deployment work by maintaining release scripts and environment notes, helping the team execute backend updates more consistently and reducing manual mistakes during repeatable deployment activities.
- Assisted with production debugging by reviewing logs, reproducing issues locally, and shipping focused low-risk patches, which improved defect turnaround while building reliable habits around structured troubleshooting and validation.
- Improved database access safety by writing clearer SQL queries and adding basic indexes for admin dashboards, which helped backend screens remain usable as record volume increased across internal operational workflows.
- Practiced incremental backend improvement by documenting reproduction steps, isolating root causes, and preferring smaller safe fixes over large rewrites, which reduced regression risk while strengthening day-to-day engineering discipline.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (*2010 – 2014)*

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.